How A Polynomial Fit Turns Into Filter Weights

A 3x3 Example, Then the General Square, Then a Circle

1 The Core Idea

The important fact is simple. Once the sample locations are fixed, the least-squares polynomial fit is a linear map from sampled pixel values to polynomial coefficients. If we only want one coefficient, for example the derivative coefficient, then that coefficient is just one fixed weighted sum of the input pixels. Those fixed numbers are the filter weights.

We can see this most clearly by starting with a tiny 3x3 example.

2 A 3x3 Toy Example

Consider a 3x3 patch centered at the origin with local coordinates

$$(x, y) \in \{-1, 0, 1\} \times \{-1, 0, 1\}. \tag{1}$$

Suppose we fit a degree-1 polynomial

$$p(x, y) = c_0 + c_1 x + c_2 y. \tag{2}$$

Here c_0 tells us the local baseline brightness, while c_1 and c_2 tell us how that brightness changes across space. Since an edge is a place where intensity changes, the derivative coefficients are the quantities we care about most. The constant term helps the polynomial fit the local patch, but the edge information is carried by the change terms.

2.1 Start With The Actual 3x3 Patch

It is easier to begin with the image patch itself rather than the design matrix. Write the local 3x3 neighborhood as

$$\boldsymbol{B} = \begin{pmatrix} I_{-1,1} & I_{0,1} & I_{1,1} \\ I_{-1,0} & I_{0,0} & I_{1,0} \\ I_{-1,-1} & I_{0,-1} & I_{1,-1} \end{pmatrix}. \tag{3}$$

This is just the actual image data in the local window. Each entry is the intensity value at one local coordinate.

2.2 Stack The Patch Into A Vector

To solve the least-squares system, we flatten those same nine values into a column vector

$$\boldsymbol{b} = \left[I_{-1,-1}, I_{0,-1}, I_{1,-1}, I_{-1,0}, I_{0,0}, I_{1,0}, I_{-1,1}, I_{0,1}, I_{1,1}\right]^\top. \tag{4}$$

The unknown polynomial coefficients are

$$\boldsymbol{z} = [c_0, c_1, c_2]^\top. \tag{5}$$

2.3 Build The Design Matrix

Each row of the design matrix is just the basis $[1, x, y]$ evaluated at one sample location. Therefore

$$\boldsymbol{A} = \begin{pmatrix} 1 & -1 & -1 \\ 1 & 0 & -1 \\ 1 & 1 & -1 \\ 1 & -1 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & -1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}. \tag{6}$$

The least-squares model is then

$$\boldsymbol{A}\boldsymbol{z} \approx \boldsymbol{b}. \tag{7}$$

2.4 Solve The Least-Squares System

The fitted coefficients are

$$\hat{\boldsymbol{z}} = (\boldsymbol{A}^\top \boldsymbol{A})^{-1} \boldsymbol{A}^\top \boldsymbol{b}. \tag{8}$$

For this particular 3x3 geometry, one can compute

$$\boldsymbol{A}^\top \boldsymbol{A} = \begin{pmatrix} 9 & 0 & 0 \\ 0 & 6 & 0 \\ 0 & 0 & 6 \end{pmatrix}, \tag{9}$$

so

$$(\boldsymbol{A}^\top \boldsymbol{A})^{-1} = \begin{pmatrix} \frac{1}{9} & 0 & 0 \\ 0 & \frac{1}{6} & 0 \\ 0 & 0 & \frac{1}{6} \end{pmatrix}. \tag{10}$$

Thus the pseudoinverse is

$$\boldsymbol{P} = (\boldsymbol{A}^\top \boldsymbol{A})^{-1} \boldsymbol{A}^\top. \tag{11}$$

The second row of $\boldsymbol{P}$ gives the weights for c_1, the x-derivative coefficient:

$$\boldsymbol{p}_{c_1}^\top = \left[-\frac{1}{6}, 0, \frac{1}{6}, -\frac{1}{6}, 0, \frac{1}{6}, -\frac{1}{6}, 0, \frac{1}{6} \right]. \tag{12}$$

So the fitted derivative coefficient is

$$\hat{c_1} = \boldsymbol{p}_{c_1}^\top \boldsymbol{b}. \tag{13}$$

Written out explicitly,

$$\hat{c_1} = \left(-\frac{1}{6}\right) I_{-1,-1} + 0 I_{0,-1} + \left(\frac{1}{6}\right) I_{1,-1} \tag{14}$$

$$+ \left(-\frac{1}{6}\right) I_{-1,0} + 0 I_{0,0} + \left(\frac{1}{6}\right) I_{1,0} \tag{15}$$

$$+ \left(-\frac{1}{6}\right) I_{-1,1} + 0 I_{0,1} + \left(\frac{1}{6}\right) I_{1,1}. \tag{16}$$

This is the crucial moment. The polynomial fit has turned into a 3x3 derivative filter:

$$\boldsymbol{K}_x = \begin{pmatrix} -\frac{1}{6} & 0 & \frac{1}{6} \\ -\frac{1}{6} & 0 & \frac{1}{6} \\ -\frac{1}{6} & 0 & \frac{1}{6} \end{pmatrix}. \tag{17}$$

The fit sounds like solving for a polynomial, but because the fit is linear, the derivative we extract is just one fixed weighted sum of the nine pixels.

3 Generalization To An N x N Square

Now suppose the support is an arbitrary square window with coordinates

$$(x_i, y_i), \quad i = 1, 2, ..., N_p, \tag{18}$$

where for a square window $N_p = N^2$.

For a polynomial of degree d, define the monomial basis vector

$$\varphi_{d(x,y)} = \left[1, x, y, \frac{x^2}{2}, xy, \frac{y^2}{2}, ...\right]^\top, \tag{19}$$

with total length

$$M = (d+1)\frac{d+2}{2}. \tag{20}$$

Then the design matrix is built exactly the same way:

$$\boldsymbol{A} = \begin{pmatrix} \varphi_{d(x_1,y_1)}^\top \\ \varphi_{d(x_2,y_2)}^\top \\ \vdots \\ \varphi_{d\left(x_{N_p},y_{N_p}\right)}^\top \end{pmatrix}. \tag{21}$$

The fitted coefficients are still

$$\hat{\boldsymbol{z}} = \left(\boldsymbol{A}^\top \boldsymbol{A}\right)^{-1} \boldsymbol{A}^\top \boldsymbol{b} = \boldsymbol{P}\boldsymbol{b}. \tag{22}$$

If the derivative coefficient we want is the entry corresponding to x, then one row of $\boldsymbol{P}$ gives a weight vector

$$\boldsymbol{p}_{f_x}^\top, \tag{23}$$

and the derivative estimate is

$$\hat{f}_x = \boldsymbol{p}_{f_x}^\top \boldsymbol{b}. \tag{24}$$

So nothing changes conceptually from 3x3 to NxN. The matrix gets larger, but the derivative is still one row of the pseudoinverse dotted with the pixel vector.

4 From A Square To A Circle

The circular case is not a new algorithm. It is the same least-squares construction with a different set of sample locations.

Instead of using every point in a square, keep only the points satisfying

$$x_i^2 + y_i^2 \leq r^2. \tag{25}$$

Now the data vector contains only the pixel values inside that circular support,

$$\boldsymbol{b} = \left[I_1, I_2, ..., I_{N_p}\right]^\top, \tag{26}$$

and the design matrix keeps only the corresponding rows

$$\boldsymbol{A} = \begin{pmatrix} \varphi_{d(x_1,y_1)}^\top \\ \varphi_{d(x_2,y_2)}^\top \\ \vdots \\ \varphi_{d\left(x_{N_p},y_{N_p}\right)}^\top \end{pmatrix}, \tag{27}$$

where the coordinates now come from the circular neighborhood rather than the full square.

The least-squares solution is unchanged:

$$\hat{\boldsymbol{z}} = \left(\boldsymbol{A}^\top \boldsymbol{A}\right)^{-1} \boldsymbol{A}^\top \boldsymbol{b}. \tag{28}$$

And again, the derivative estimate is one row of the pseudoinverse:

$$\hat{f}_x = \boldsymbol{p}_{f_x}^\top \boldsymbol{b}. \tag{29}$$

So the circle does not change the logic at all. It only changes which sample coordinates appear as rows of $\boldsymbol{A}$.

5 Why This Matters For WVF

The WVF does two additional things.

First, it rotates the local coordinates so that the derivative is taken in the candidate normal direction:

$$x_{i'} = \Delta x_i \cos\theta + \Delta y_i \sin\theta, \tag{30}$$

$$y_{i'} = -\Delta x_i \sin\theta + \Delta y_i \cos\theta. \tag{31}$$

Second, it uses a circular support instead of a square one.

But once those rotated circular sample locations are fixed, the exact same least-squares logic applies:

$$\hat{\boldsymbol{z}} = \boldsymbol{P}_\theta \boldsymbol{b}, \tag{32}$$

and the WVF weight vector is just the row of $\boldsymbol{P}_\theta$ corresponding to the normal derivative coefficient:

$$\hat{f}_{x'} = \boldsymbol{p}_{f_{x'}}^\top \boldsymbol{b}. \tag{33}$$

That row is the WVF stencil.

6 The Short Version

The shortest correct explanation is this.

The polynomial fit is a linear map from sampled pixel values to polynomial coefficients. Because the derivative coefficient is one entry of the fitted coefficient vector, it is also a linear map of the sampled pixels. Any linear map from pixels to a scalar can be written as one fixed weighted sum. Those fixed numbers are the filter weights.

So the WVF weights are not something added after the polynomial fit. They are exactly the polynomial fit, collapsed into the one row needed to extract the derivative.